

FLEX — ablation studies

Fourteen families, every number read from the measurement files. The main experiment throughout is RECIPE512 at epoch 9, 256 px tiles, 53 CTC intra frames, 0.1 dB, unless a section says otherwise. Where a family was measured on the dumped per-cell tables rather than a real decode it says so, because the tables carry no tiling penalty and sit optimistically.

1. Decode geometry — where the saving comes from

Rules out: that the saving is an early-exit effect alone. Most of it is, but a measurable part is the cost of cutting the frame into tiles, and that part is recoverable without touching the ladder.

variant	mean	qp0	qp16	qp32	qp48	qp63	Δ
tiled 256 px (the paper)	27.64%	33.69	30.14	26.07	24.78	23.52	
per-position, 256 px cells	31.51%	36.92	33.99	30.69	28.69	27.26	+3.87
per-position, 64 px cells	33.27%	38.17	35.56	32.54	30.76	29.35	+1.77

Tiled decoding pays a **floor**: every 3×3 at a tile border reads padding instead of a neighbour, and that costs quality before any tile has exited early. Per-position depth removes the cut entirely — each position runs to its own depth and the receptive field is handled by a max-plus dilation rather than by a boundary — so the floor drops from 0.048 dB to 0.020 dB at the top rate and the same budget buys more.

signalled_RECIPE512_ctc53.json + granularity_ctc53_fixed.json · epoch 4

2. Granularity — how fine the allocation should be

Rules out: that finer is always better. It is better in multiply-accumulates and worse once a real kernel is charged (section 9).

cell size	mean saving	band cost
256 px	31.51%	0.33 points
128 px	32.66%	0.73 points
64 px	33.27%	1.56 points
32 px	33.59%	2.75 points

The **band** is what a finer allocation spends its winnings on: positions that keep computing only to serve a deeper neighbour. It is 0.33 points at 256 px and 2.75 at 32 px, and it is why the curve flattens rather than continuing.

flexplus/results/granularity_ctc53_fixed.json

3. The clamp — are the two lowest exits worth reaching

Rules out: that removing the clamp is an independent knob. It is not: under tiling the first exits are not cheaper, so the clamp can only be lifted together with the tiling.

split depth	mean saving	ceiling
j = 2	33.32%	40.08%
j = 0	36.02%	69.89%

In the tiled architecture the first j groups run over the whole frame before patchify, so exits 0, 1 and 2 all cost 0.5716 of a decode — identical. Choosing a shallower one there buys nothing and loses quality. Only per-position decoding, which has no shared prefix, makes them genuinely cheap, and then the ceiling rises from 40.1% to 69.9%.

flexplus/results/cells_ctc64.npz · epoch 4

4. Baselines — what predicts the exit-error curve

Rules out: that a learned router is needed for the prediction. It is not: a parameter-free rule reading the entropy coder's own bit count beats it.

model	mean saving	of the oracle
oracle	33.32%	100.0%
gbm_shape	30.70%	92.1%
ridge	30.61%	91.9%
raterank	30.03%	90.1%
gbm	27.00%	81.0%
flat	15.10%	45.3%

The oracle is the exhaustive per-tile search and bounds what any router can reach. **flat** is the control with no adaptivity at all — one curve for every tile — and the gap between it and the oracle is the whole of what routing can win.

flexplus/results/router_fit.json · 64 px cells, j = 2

5. Router inputs — which group carries the signal

Rules out: that the head needs all of its inputs. One group at a time is zeroed after its projection, so every variant has the same architecture, the same parameter count and the same optimiser state, and the only difference is how much the head may know.

inputs live	agreement with the oracle	best constant
stem,qp	77.4%	38.2%
stem,latent,scales,bits,qp	77.0%	38.2%
latent,qp	74.5%	38.2%
scales,qp	71.1%	38.2%
bits,qp	58.5%	38.2%
qp	41.3%	38.2%

qp is the control that matters: it is one number per frame, the same for every tile in it, so it cannot in principle separate two tiles of one frame. Whatever agreement it reaches is what is available with no per-tile information at all.

FLEX-UF/results/router_ablation_*_e4.json

6. The closed-form refit — is it the refit or the conditioning

Rules out: that re-solving the adapters is what helps. Two thirds of the gain is lost when the same solve runs on all tiles, and all of it is lost when it runs on survivors chosen at the wrong multiplier.

arm	mean saving	against baseline
baseline, epoch 9	28.15%	—
survivors, matched lambda	28.80%	+0.64
all tiles (control)	28.34%	+0.19
survivors, WRONG lambda (control)	28.14%	-0.01

Each adapter ends in a pointwise linear map, and the target is known — the deepest exit's raw feature, which is what the shared head was fitted to consume. So the solve is a ridge regression, in the manner of AdaRound and BRECCQ, and it needs no gradient step. 443,520 parameters change, 0.98% of the decoder, two minutes on one card.

The third row is the control that carries the argument. Same method, same data, same number of parameters, and the conditional distribution wrong: the gain vanishes to −0.01. What the method does is not fit; it is fit *where the adapter is used*.

flexplus/results/signalled_CALEX_A_*.json · epoch 9

7. Fine-tuning — does gradient descent add anything

Rules out: that the adapters are undertrained. With the anchor on the released decoder and the adapters at the main run's own learning rate, six thousand steps return the baseline exactly, with the usage prior and without it.

arm	mean saving	against baseline
baseline, no fine-tune	28.32%	+0.00
6000 steps, usage-matched weights	28.32%	-0.01
6000 steps, uniform weights (control)	28.32%	-0.01
closed-form refit, no training	28.74%	+0.42

This is the negative result that explains the positive one. The adapters have converged — for an **average** distribution — and stochastic gradient descent on OpenImages cannot be aimed away from it: every step sees every tile, and the multiplier matching that makes the closed form work has no analogue in the training loop. The closed form can state the conditional directly.

A first attempt lost 3.52 points with the prior and 8.21 without it. Both losses were the harness, not the idea: the anchor had been pinned to the run's own starting checkpoint, where it is computed on the full-frame path a frozen backbone cannot move, so anchor_mse was identically zero and nothing held the tiled deepest path. Corrected, both arms return to the baseline.

flexplus/results/signalled_perframe_*.json · epoch 9

8. Where the multiplier is chosen — the guarantee

Rules out: that a per-frame guarantee costs saving. It raises it, because a single multiplier solves a Lagrangian on pooled squared error while the constraint is on a mean of logarithms.

multiplier	mean saving	over budget	95th pct	worst
one lambda per rate	28.15%	134 / 265	0.167 dB	0.261 dB
one lambda per frame	28.32%	1 / 265	0.100 dB	0.141 dB

Measured on the deployed tiled path, one real decode per bisection step, and the multiplier returned is by construction one whose delivered decibel is at or under the budget — the bound holds by the solver rather than by its convergence. An earlier version bisected a cheap table and corrected six times, which left 17 of 53 frames above the budget at qp0: a guarantee measured by a solver that does not guarantee.

flexplus/results/guarantee_tiled_e9.json · epoch 9

9. Implementability — what a real kernel would keep

Rules out: that the multiply-accumulate count is the saving. A block-sparse kernel must run any B×B tile containing one needed position, and the finer allocation loses more to that than the coarse one.

kernel granularity	256 px cells	64 px cells
ideal, per position	31.51%	33.32%
B = 2 feature positions (16 px)	31.28%	32.33%
B = 4 feature positions (32 px)	30.88%	30.70%
B = 8 feature positions (64 px)	29.94%	27.63%
B = 16 feature positions (128 px)	27.91%	21.54%
B = 32 feature positions (256 px)	24.17%	8.60%

At B = 8 the coarse allocation keeps 30.06% against the fine one's 27.99%, reversing the ideal ranking. Granularity has an optimum once implementability is charged, and it is coarser than the multiply-accumulate optimum.

flexplus/results/mask_structure_*.all.json

10. Can the decoder shorten its own tail

Rules out: that the decoder-side tail is made of near-ties. Deepening every tile whose best two exits score within a margin does not move the worst frame at any margin.

margin δ	mean saving	worst frame	over 0.1 dB
0.00	24.17%	0.475 dB	116 / 265
0.10	24.17%	0.475 dB	116 / 265
0.25	24.86%	0.475 dB	115 / 265
0.40	25.54%	0.475 dB	114 / 265
0.55	25.61%	0.475 dB	114 / 265
0.70	25.61%	0.475 dB	114 / 265
0.85	25.61%	0.475 dB	114 / 265
0.95	25.61%	0.475 dB	114 / 265

The tail is not indecision; it is frames where the surrogate is confidently wrong, and no tie-break reaches those. Letting the decoder throttle itself on its own predicted decibel does shorten it, but only by giving up most of the saving:

predicted target	mean saving	worst frame
0.100 dB	18.14%	0.315 dB
0.085 dB	15.75%	0.315 dB
0.070 dB	13.14%	0.315 dB
0.060 dB	10.40%	0.216 dB
0.050 dB	8.77%	0.208 dB
0.040 dB	6.99%	0.192 dB

What works costs sixteen bits. The encoder already runs a replica of the decoder's predictor in configuration C, so it can evaluate the **true** frame decibel of the allocation the decoder will make and bisect on it, then send that one scalar: 264 of 265 frames inside the budget, worst 0.114 dB, and the saving rises from 24.17% to 25.06%. The map is still never transmitted; only its price is.

flexplus/results/safe_routing.json · per-cell tables, no tiling penalty

11. Generalisation — an image benchmark, not video frames

Rules out: that the method transfers uniformly. It does not: at the top rate the tiling floor alone exceeds the budget on a third of Kodak.

	mean saving	images inside 0.1 dB	mean tiling floor
Kodak, qp 0	33.93%	24 / 24	0.0220 dB
Kodak, qp 32	25.83%	24 / 24	0.0345 dB
Kodak, qp 63	5.87%	16 / 24	0.0894 dB

The infeasibility is the floor, not the allocation: delivered decibel equals floor for those images, and no granularity rescues them. It is the second, independent argument for removing the tiling — the floor is not merely a tax on saving, it is what makes a per-frame guarantee unreachable on hard content.

flexplus/results/kodak_*.json · epoch 4

12. The operating point

Rules out: that 0.1 dB is a property of the system. It is a choice, and the saturation structure says what it costs.

budget	mean saving	frames already saturated
0.100000 dB	28.74%	0 / 265
0.121317 dB	31.33%	65 / 265

A saturated frame is one whose ceiling — every tile at the shallowest exit it may take — already sits under the budget, so the tolerance above that point is spent on nothing. On the deployed tiled path the ceiling is content-dependent and spreads from 0.017 to 0.860 dB, so saturation arrives gradually rather than at one sharp point.

flexplus/results/signalled_perframe_*.json

13. Training length

Rules out: that more training is the remaining lever.

epoch	mean saving
0	30.28%
1	25.28%
2	24.04%
3	25.84%
4	27.64%
5	27.89%
6	28.07%
7	27.91%
8	28.14%
9	28.15%
10	28.13%

Epoch 9 to 10 is -0.02 at a learning rate of $1e-6$. The ladder has converged, and the improvements that remain are not training improvements — which is what sections 6 and 7 measure.

signalled_RECIP512_*

14. Is the usage prior a property of the test set

Rules out: that the closed-form refit borrows from the test half. Two disjoint halves of the video set agree to 0.029 total variation, while a different domain differs by 0.283.

source of the prior	e0	e1	e2	e3	e4	e5
CTC half A (27 sequences)	0.0%	0.0%	70.3%	22.2%	4.8%	2.7%
CTC half B (26 sequences)	0.0%	0.0%	71.9%	20.2%	3.8%	4.1%
CTC, all 53	0.0%	0.0%	71.0%	21.1%	4.3%	3.6%
OpenImages (the training domain)	0.0%	0.0%	44.8%	27.8%	2.2%	25.2%

KL between the halves is 0.0048 nat; between the training domain and the test domain it is 0.3474. So the prior can be estimated from held-out material and 27 sequences suffice — but it has to be held-out material of the **deployment** domain, because photographs give a materially different vector than video frames do.

flexplus/results/prior_stability.json